# Solving 2048: A Comparative Study of Classical Reinforcement Learning and LLM-Guided RLVR

Shriman Raghav Srinivasan  Gautham Ramkumar
MS Robotics, Northeastern University, Boston, MA
srinivasan.shrim@northeastern.edu  ramkumar.g@northeastern.edu

## Abstract

We present a systematic comparison of classical reinforcement learning and Reinforcement Learning with Verifiable Rewards (RLVR) on the 2048 puzzle game. Six classical agents (Deep Q-Network [DQN], Proximal Policy Optimization [PPO], Advantage Actor-Critic [A2C], Quantile Regression DQN [QR-DQN], discrete Soft Actor-Critic [SAC], and Semi-gradient SARSA with linear function approximation) are benchmarked against Qwen2.5 language models (0.5B and 1.5B parameters) fine-tuned via Group Relative Policy Optimization (GRPO). All deep classical agents share an identical three-layer CNN backbone (330K parameters) to isolate algorithmic effects. We evaluate at three levels: 1M training steps, 5M training steps, and unlimited-attempt hunt mode. DQN achieves the highest classical score (avg 7,744 at 5M steps; tile 2048 in hunt mode), while Semi-gradient SARSA with only 120 parameters outperforms all deep on-policy methods, demonstrating that hand-crafted domain features can outperform end-to-end CNN learning under sparse-reward conditions. GRPO-trained LLMs reach avg 2,348 (Qwen2.5-1.5B) vs. DQN's 7,744, show model-scale dependence (3× parameters → 3.3× score), but classical agents remain ∼400× faster at inference. We release a live dashboard and complete codebase for reproducibility: https://github.com/shrirag10/2048-RLVR.

## 1. Introduction

The tile-sliding puzzle 2048 was introduced in 2014 by Gabriele Cirulli and quickly attracted attention from the AI community. On every turn the player selects one of four cardinal directions; all tiles slide accordingly, identical adjacent tiles merge (doubling their value), and a fresh 2 or 4 tile spawns at a uniformly random empty cell. The game terminates when no legal move remains.

Despite its compact 4×4 board, 2048 poses fundamental challenges for learning agents. The state space is vast ($\sim 10^{19}$ configurations), rewards are sparse and delayed (hundreds of preparatory moves may earn nothing before a single high-value merge), and stochastic tile spawns make planning difficult. These properties make 2048 an ideal benchmark for comparing fundamentally different learning paradigms.

Recently, a new paradigm has emerged: Reinforcement Learning with Verifiable Rewards (RLVR). Instead of training a neural network policy via environment interaction, RLVR fine-tunes large language models (LLMs) to generate chain-of-thought reasoning validated by deterministic reward functions [1, 2]. While RLVR has yielded breakthroughs in mathematical reasoning, its application to spatial strategy games remains largely unexplored: symbolic math operates over linear token sequences, while grid games require models to maintain consistent 2D spatial relationships across positional labels, a distinct and harder generalization target.

This paper asks three questions: **(Q1)** How do six classical RL algorithms compare on 2048 when architectural confounds are removed? **(Q2)** Can small LLMs (0.5B–1.5B parameters) acquire 2048 strategy through GRPO alone, without using proprietary LLM APIs? **(Q3)** What are the fundamental trade-offs between classical RL's throughput and RLVR's interpretability?

Our contributions include:

1. A controlled benchmark of six classical agents and two GRPO-LLM agents on a unified 2048 environment with clear 1M/5M/hunt-mode evaluation tiers.
2. Demonstration that Semi-gradient SARSA (Sutton & Barto, Algorithm 10.1 [3]) with 120 parameters outperforms all deep on-policy methods with 330K+ parameters.
3. A GRPO training pipeline for locally deployed LLMs with analysis of model-scale dependence and spatial reasoning failures.
4. Evidence that on-policy PPO actively degrades with extended training on 2048, losing 8% score from 1M to 5M steps.

## 2. Background

### 2.1 Markov Decision Process Formulation

The 2048 game is modeled as a finite-horizon MDP $(\mathscr{S}, \mathscr{A}, P, R, \gamma)$ where:

- $\mathscr{S}$: Board states, encoded as 16-channel binary tensors $\mathbf{x} \in \{0,1\}^{16\times4\times4}$ for CNN agents, or ASCII text for LLM agents.
- $\mathscr{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$: Four discrete ac-

tions.

- $P(s'|s,a)$: Deterministic tile-sliding with stochastic spawn (90% tile-2, 10% tile-4 at random empty cell).
- $R(s,a,s')$: Score-delta plus milestone bonuses (+50 at 256, +100 at 512, +200 at 1024) and $-1$ penalty for invalid moves (directions that produce no tile movement).
- $\gamma = 0.99$: Discount factor. For games that can exceed 500 moves, the effective planning horizon ($\sim$100 steps) truncates the influence of terminal rewards, making reward shaping via milestone bonuses essential.

### 2.2 Value-Based Methods

Value-based approaches estimate the action-value function $Q^\pi(s,a) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$ and act greedily. DQN [4] parameterizes $Q$ as a neural network and stabilizes learning via experience replay and a frozen target network. QR-DQN [5] extends DQN by modeling the full return distribution via quantile regression.

### 2.3 Policy-Gradient Methods

Policy-gradient methods directly optimize the policy $\pi_\theta(a|s)$. PPO [6] uses a clipped surrogate objective with Generalized Advantage Estimation (GAE). A2C [7] is a synchronous actor-critic using $n$-step returns. SAC [8] augments the RL objective with an entropy bonus, encouraging exploration.

### 2.4 Linear Function Approximation

Sutton & Barto (§9.3, [3]) describe linear function approximation where $\hat{q}(s,a,\mathbf{w}) = \mathbf{w}_a^\top \phi(s)$, with $\phi(s)$ a hand-crafted feature vector. Semi-gradient SARSA (Algorithm 10.1) updates weights using only the gradient of the prediction, not the bootstrap target. Under linear FA, semi-gradient SARSA converges to a TD fixed point with bounded approximation error (§9.4).

### 2.5 RLVR and Group Relative Policy Optimization

RLVR trains LLMs using rewards from deterministic verifiers rather than human feedback [1]. Group Relative Policy Optimization (GRPO) [9] eliminates the critic network by normalizing advantages within a group of $G$ sampled completions per prompt:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \varepsilon} \tag{1}$$

This replaces the critic's state-dependent baseline with a sample-based, prompt-specific baseline.

## 3. Related Work

**Classical approaches to 2048.** Szubert & Jaśkowski [10] use temporal-difference learning with n-tuple networks, achieving tile 65536 via tabular value approximation over hand-designed board patterns. Saligram et al. [11] apply DQN with delayed-reward shaping. We exclude expert-designed tabular methods such as n-tuple TD from our baselines because our focus is on general-purpose deep RL algorithms that require no 2048-specific representation design; Semi-gradient SARSA with 30 engineered features serves as our closest proxy.

**Deep RL for games.** Mnih et al. [4] demonstrated human-level Atari play with DQN. Subsequent work introduced distributional RL [5], entropy-regularized policies [8], and trust-region methods [6]. We evaluate all four paradigms on a single game with matched architectures.

**LLMs for game-playing.** Hu et al. [12] benchmark LLMs on diverse games including 2048, finding that even large models struggle with spatial reasoning. Our work extends this by *training* LLMs via GRPO rather than prompting pretrained models, and directly comparing the trained LLM against classical RL agents under controlled conditions.

**RLVR.** DeepSeek-R1 [1] showed that GRPO can incentivize reasoning in LLMs using verifiable rewards. DeepSeek-Math [9] applied GRPO to mathematical reasoning. We extend RLVR to spatial strategy games and quantify the gap between RLVR and classical approaches.

## 4. Methodology

### 4.1 Environment

We implement a custom 2048 Gymnasium [13] environment. For CNN agents, the board is encoded as a 16-channel binary tensor (Eq. 2):

$$x_{k,i,j} = \mathbb{1}[B_{i,j} = 2^k], \quad k = 0, \ldots, 15 \tag{2}$$

For LLM agents, the board is serialized as a labeled ASCII grid with current score and max tile.

### 4.2 Shared CNN Backbone

Every deep classical agent uses the same three-layer convolutional feature extractor: Conv2D($16 \to 128$, $2\times2$) $\to$ ReLU $\to$ Conv2D($128 \to 128$, $2\times2$) $\to$ ReLU $\to$ Conv2D($128 \to 128$, $2\times2$) $\to$ ReLU $\to$ FC($128 \to 256$). This produces a 256-dimensional feature vector; each algorithm appends its own task-specific heads. The shared backbone ensures performance differences reflect the *learning algorithm*, not network capacity.

### 4.3 Classical Agents

#### *4.3.1 DQN (Custom PyTorch).*

Our DQN uses a custom PyTorch implementation rather than Stable-Baselines3 (SB3) because SB3's DQN lacks direct Q-value masking for invalid actions (it only supports action-space wrappers). Our implementation masks invalid Q-values to $-\infty$ *inside the forward pass*, enabling proper $\varepsilon$-greedy selection over legal moves only.

Hyperparameters: replay buffer 100K, batch size 64, $\varepsilon$: $1.0 \to 0.01$ over 100K steps, target sync every 1K steps, learning rate $10^{-4}$ (Adam), gradient clipping 1.0.

**Algorithm 1** Deep Q-Network (DQN) for 2048

Initialize replay buffer $\mathscr{D}$ with capacity $100,000$
Initialize Q-network $Q_\theta$ and target network $Q_{\theta'} \leftarrow Q_\theta$
**for** step $t = 1$ to $T$ **do**
  With probability $\varepsilon$, select random valid action $a_t$
  Else select $a_t = \arg\max_a Q_\theta(s_t, a)$
  Execute $a_t$, observe reward $r_t$, next state $s_{t+1}$
  Store transition $(s_t, a_t, r_t, s_{t+1})$ in $\mathscr{D}$
  **if** $t \geq$ batch_size **then**
    Sample mini-batch of transitions from $\mathscr{D}$
    Compute target $y_j = r_j + \gamma \max_{a'} Q_{\theta'}(s_{j+1}, a')$
    Update $\theta$ by minimizing Huber loss $(y_j - Q_\theta(s_j, a_j))^2$
  **end if**
  **if** $t$ mod target_freq $== 0$ **then**
    $\theta' \leftarrow \theta$
  **end if**
  Decay $\varepsilon$ linearly
**end for**

#### 4.3.2 SAC - Discrete (Custom PyTorch).

**Algorithm 2** Discrete Soft Actor-Critic (SAC)

Initialize twin Q-networks $Q_{\theta_1}, Q_{\theta_2}$, targets $Q_{\theta'_1}, Q_{\theta'_2}$
Initialize actor network $\pi_\phi$, temperature parameter $\alpha$
Initialize replay buffer $\mathscr{D}$
**for** step $t = 1$ to $T$ **do**
  Sample $a_t \sim \pi_\phi(\cdot|s_t)$ and execute
  Store $(s_t, a_t, r_t, s_{t+1})$ in $\mathscr{D}$
  Sample mini-batch from $\mathscr{D}$
  $V(s_{t+1}) = \sum_a \pi_\phi(a|s_{t+1})[\min_{i=1,2} Q_{\theta'_i}(s_{t+1}, a) - \alpha \log \pi_\phi(a|s_{t+1})]$
  $y_t = r_t + \gamma V(s_{t+1})$
  Update critics $\theta_i$ to minimize $(Q_{\theta_i}(s_t, a_t) - y_t)^2$
  Update actor $\phi$ to minimize $\sum_a \pi_\phi(a|s_t)[\alpha \log \pi_\phi(a|s_t) - \min_{i=1,2} Q_{\theta_i}(s_t, a)]$
  Update $\alpha$ to minimize $\alpha(\log \pi_\phi(a_t|s_t) + \bar{\mathscr{H}})$
  Soft update target networks $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$
**end for**

Standard SAC [8] targets continuous actions. We implement the discrete adaptation of Christodoulou [14] with categorical policies, twin Q-networks, and automatic temperature tuning. This required custom code because SB3 does not natively support discrete SAC with invalid action masking in the policy distribution.

Hyperparameters: replay buffer 200K, batch size 256, Polyak $\tau = 0.005$, learning rate $10^{-4}$, warmup 10K steps, target entropy $0.4 \cdot \log|\mathscr{A}|$.

#### 4.3.3 PPO - MaskablePPO (Stable-Baselines3).

We use SB3's MaskablePPO [15], which natively supports action masking via environment-provided masks.

Hyperparameters: rollout 512 steps $\times$ 8 parallel envs, batch size 128, 4 epochs per update, learning rate $2\times10^{-4}$, $\gamma$=0.995, clip $\varepsilon = 0.2$, entropy coefficient $c_e = 0.05$.

#### 4.3.4 A2C (Stable-Baselines3).

A2C [7] is on-policy actor-critic without PPO's clipping mechanism.

Hyperparameters: 5-step returns, learning rate $7\times10^{-4}$, entropy coefficient 0.01, value coefficient 0.5.

#### 4.3.5 QR-DQN (SB3-Contrib).

QR-DQN [5] models the return distribution via $N$=50 quantiles, outputting 4×50=200 values.

Hyperparameters: buffer 100K, batch 64, learning rate $10^{-4}$, target sync every 1K steps, $\varepsilon$: $1.0 \to 0.05$ over 10% of training.

#### 4.3.6 Semi-gradient SARSA (Custom NumPy).

**Algorithm 3** Semi-gradient SARSA with Linear FA

Initialize weight vector $\mathbf{w} = \mathbf{0} \in \mathbb{R}^{30}$
**for** episode $e = 1$ to $E$ **do**
  Extract features $\phi(S)$ from initial state $S$
  Choose $A \sim \varepsilon$-greedy$(\mathbf{w}^\top \phi(S, \cdot))$
  **repeat**
    Take action $A$, observe $R, S'$
    **if** $S'$ is terminal **then**
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R - \mathbf{w}^\top \phi(S,A)]\phi(S,A)$
    **else**
      Choose $A' \sim \varepsilon$-greedy$(\mathbf{w}^\top \phi(S', \cdot))$
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \mathbf{w}^\top \phi(S', A') - \mathbf{w}^\top \phi(S,A)]\phi(S,A)$
      $S \leftarrow S', A \leftarrow A'$
    **end if**
  **until** $S'$ is terminal
**end for**

Following Sutton & Barto, Algorithm 10.1 [3], we implement episodic semi-gradient SARSA with linear function approximation. The agent maintains $\mathbf{w} \in \mathbb{R}^{4\times30}$ (120 parameters total) mapping a 30-dimensional feature vector $\phi(s)$ to Q-values:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}_a^\top \phi(s) \tag{3}$$

The feature vector $\phi(s) \in \mathbb{R}^{30}$ encodes: 16 log-tile values $\log_2(B_{ij}+1)/16$, empty-cell ratio $|E|/16$, merge potential (count of adjacent equal pairs, normalized), 4 monotonicity scores (one per row/column direction, $\sum|\log_2 B_i - \log_2 B_{i+1}|$), max-tile indicator $\log_2(\max B)/16$, corner-max bonus $\mathbb{1}[\max B \in \text{corners}]$, mean/std/max of log-tile values, smoothness $-\sum|\log_2 B_i - \log_2 B_j|$ over adjacent pairs, snake-pattern score (monotonic zigzag through cells), and edge-tile sum $\sum_{(i,j)\in\partial} \log_2 B_{ij}$. Features are normalized to $[0,1]$ via division by their theoretical maxima. The semi-gradient update

rule (§10.1) differentiates only through the prediction:

$$\mathbf{w}_{a_t} \leftarrow \mathbf{w}_{a_t} + \alpha\delta_t\phi(s_t), \quad \delta_t = r_t + \gamma\hat{q}(s_{t+1}, a_{t+1}) - \hat{q}(s_t, a_t) \tag{4}$$

Hyperparameters: step-size $\alpha = 10^{-3}$, $\varepsilon$-decay 0.99995/step, $\varepsilon_{\min} = 0.05$.

### 4.4 GRPO-Trained LLM Agent

#### *4.4.1 Why Local GRPO, Not an LLM API?*

Commercial LLM APIs do not expose the per-token log-probabilities or gradient computations required for GRPO's clipped policy update. Local deployment additionally ensures deterministic training under fixed seeds and avoids silent model-version changes that would undermine reproducibility.

#### *4.4.2 Model and Training.*

**Algorithm 4** Group Relative Policy Optimization (GRPO)

Initialize LLM policy $\pi_\theta$ and reference model $\pi_{\text{ref}} \leftarrow \pi_\theta$
**for** iteration $k = 1$ to $K$ **do**
  Sample prompt $q$ (board state)
  Generate $G$ completions $\{o_1, \ldots, o_G\}$ using $\pi_{\text{old}}$
  Compute reward $r_i$ for each completion using environment rules
  Compute group advantages: $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$
  Compute policy ratio: $\rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\text{old}}(o_i|q)}$
  Compute KL divergence: $\mathbb{D}_{\text{KL}} = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - 1$
  Update $\theta$ by maximizing surrogate objective:
  $\mathscr{L}_{\text{GRPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\Big[\min\big(\rho_i A_i, \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon)A_i\big)$
    $-\beta\mathbb{D}_{\text{KL}}\Big]$
**end for**

We fine-tune Qwen2.5-0.5B-Instruct (494M params) and Qwen2.5-1.5B-Instruct (1.8B params) [16] using 4-bit NF4 quantization via Unsloth [17] with LoRA adapters ($r$=16, $\alpha$=16). The GRPO loss follows Equation 1 with a KL penalty:

$$\mathscr{L}_{\text{GRPO}}(\theta) = \mathbb{E}_q\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big(L_t^{\text{clip}} - \beta D_{\text{KL}}\Big)\right] \tag{5}$$

where $L_t^{\text{clip}} = \min(r_t^i\hat{A}_i, \text{clip}(r_t^i, 1\pm\varepsilon)\hat{A}_i)$ and $\beta$=0.04.

#### *4.4.3 Reward Scheduling.*

The reward function uses three stages to avoid cold-start collapse: (1) format compliance (XML tags, weight 0.5), (2) direction validity (weight 0.5), (3) game reward (Δscore / max_tile, weight 2.0). Quality and length components were removed after the model reward-hacked the length penalty by generating filler text.

**Table 1:** Classical agent performance at the 1M-step checkpoint, evaluated over the last 100 training episodes. DQN dominates across all metrics. Semi-gradient SARSA achieves second place with 2,750× fewer parameters.

| Agent | Avg Score | Max Score | Max Tile | 256+ | 512+ | 1024+ | Impl. |
|---|---|---|---|---|---|---|---|
| DQN | **6,384** | **14,050** | **1024** | 92% | 67% | 10% | Custom |
| SARSA | 2,455 | 10,208 | 1024 | 45% | 9% | 1% | NumPy |
| A2C | 1,205 | 2,856 | 256 | 14% | 0% | 0% | SB3 |
| SAC | 1,132 | 4,814 | 512 | 6% | 1% | 0% | Custom |
| PPO | 1,038 | 3,040 | 256 | 7% | 0% | 0% | SB3 |
| QR-DQN | 782 | 2,176 | 256 | 2% | 0% | 0% | SB3 |

## 5. Experiments

### 5.1 Setup

All experiments ran on a single NVIDIA RTX 4060 Laptop GPU (8 GB VRAM), Ubuntu 22.04, CUDA 12.6. LLM agents (Qwen2.5-0.5B and 1.5B) were trained for 500 GRPO steps over a dataset of 2,000 board-state prompts for 3 epochs (0.5B: ∼41 minutes; 1.5B: ∼76 minutes). Classical agents were additionally evaluated in "hunt mode": unlimited stochastic episodes ($\varepsilon$=0.05) using the 1M-step checkpoint until reaching tile 2048 or exhausting 3,000 attempts.

**Training Step Accounting.** All classical agents were trained for 5M total environment interactions. DQN uses a single environment ($n_{\text{envs}}$=1); SB3-based agents (PPO, A2C, QR-DQN, SAC) use 8 parallel environments, but SB3's `total_timesteps` parameter counts total interactions across all environments, so 5M steps equals 5M interactions for all agents. The asymmetry lies in gradient updates: DQN performs one gradient step per interaction (up to 5M updates), while PPO with rollout length 512 and 8 environments performs one batch update per 512×8=4,096 interactions (∼1,220 updates over 5M steps). Off-policy replay thus provides both more gradient updates and the ability to revisit past transitions.

**Statistical caveat.** All reported scores are point estimates from a single training run per agent, averaged over the final 100 training episodes. Multi-seed evaluation was not feasible within the project timeline. Large gaps (DQN vs. PPO, > 7,000 score points) are almost certainly robust; smaller gaps (A2C vs. SAC, within 700 score points) should be interpreted as suggestive rather than definitive.

### 5.2 Classical RL Results: 1M Steps

Table 1 presents all six classical agents evaluated over their last 100 training episodes at the 1M checkpoint.

DQN dominates at 1M steps, reaching tile 1024 in 10% of games. The 8× gap over PPO (6,384 vs. 1,038) is attributable to experience replay: DQN's 100K-transition buffer enables

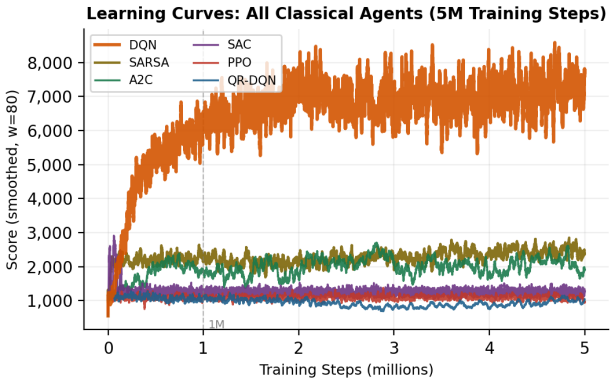


**Figure 1:** Smoothed learning curves for all six classical agents over 5M training steps. DQN exhibits monotonic improvement throughout, while PPO and QR-DQN plateau and subsequently degrade after approximately 1M steps. The dashed vertical line marks the 1M checkpoint used for hunt-mode evaluation.

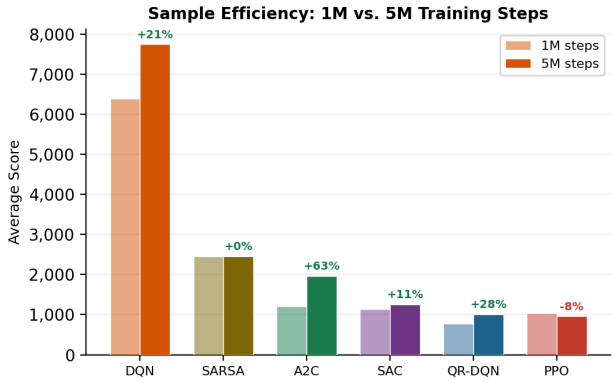


**Figure 2:** Sample efficiency comparison: average score at 1M vs. 5M training steps. Percentage labels indicate the change from the 1M checkpoint. DQN benefits most from additional training; PPO and QR-DQN degrade.

**Table 2:** Classical agent performance at 5M training steps. The Δ column shows the percentage change in average score from the 1M checkpoint (Table 1). DQN is the only agent exhibiting strong positive scaling.

| Agent | Avg Score | Max Score | Max Tile | 512+ Rate | 1M→5M Δ | Verdict |
|---|---|---|---|---|---|---|
| DQN | **7,744** | **39,314** | **2048** | **76%** | **+21%** | Strong scaling |
| SARSA | 2,456 | 11,436 | 1024 | 6% | <1% | Convergence |
| A2C | 1,964 | 9,640 | 1024 | 6% | +63% | Modest gain |
| SAC | 1,255 | 7,882 | 512 | <1% | +11% | Slow |
| QR-DQN | 1,003 | 4,508 | 512 | 0% | +28% | Modest gain |
| PPO | 957 | 4,900 | 512 | 0% | **−8%** | Degradation |

repeated updates on rare high-scoring trajectories, amplifying sparse reward signals (S&B §8.2). On-policy methods discard each trajectory after a single update, making them fundamentally sample-inefficient in reward-sparse environments.

Semi-gradient SARSA achieves second place (avg 2,455, max tile 1024) with only 120 parameters (2,750× fewer than DQN's 330K). Its hand-crafted feature vector encodes monotonicity, corner strategies, merge potential, and snake patterns, providing the "strong inductive bias" that Sutton & Barto advocate (§9.5).

Figure 1 presents the learning curves for all six agents over the full 5M-step training horizon, illustrating the divergence in scaling behavior that emerges after the 1M checkpoint.

### 5.3 Classical RL Results: 5M Steps

Table 2 shows the same agents at the 5M-step checkpoint. The 1M→5M scaling analysis reveals three distinct behavioral regimes.

Three distinct scaling regimes emerge. DQN exhibits strong positive scaling (+21%), consistent with off-policy replay continuing to benefit from additional data accumulation: the replay buffer serves as a reservoir of high-scoring trajectories that are revisited thousands of times, a mechanism unavailable to on-policy methods. SARSA shows negligible improvement (<1% gain), consistent with the bounded approximation error under linear function approximation (S&B §9.4); the 120-parameter linear model lacks the representational capacity for the complex tile interactions required beyond 1024. PPO exhibits performance degradation (−8%), developing degenerate tile-sliding loops in which the policy repeatedly selects the same two directions. A2C shows the strongest relative improvement (+63%), suggesting that the simpler actor-critic update may be more stable than PPO's clipped objective on this task.

Figure 2 quantifies these scaling behaviors as a grouped bar chart, making the divergence between off-policy and on-policy methods visually apparent.

### 5.4 Mechanism Analysis of DQN Performance

In many continuous control benchmarks, PPO and SAC outperform DQN due to entropy regularization and trust-region guarantees. On 2048, however, DQN achieves the highest score across all evaluation tiers. We identify four structural properties of the 2048 domain that favor DQN's design.

**Experience replay as a sparse-reward amplifier.** On-policy methods (PPO, A2C) use each trajectory exactly once before discarding it. DQN's 100K-transition replay buffer [4] enables repeated updates on rare high-scoring trajectories. At 5M steps with batch size 64 and a 100K buffer, the expected replay count per transition is $(5\text{M} \times 64)/100\text{K} \approx 3{,}200$. This constitutes a 3,200× amplification of the sparse reward signal relative to on-policy methods, which process each transition exactly once (S&B §8.2).

**Target network stability on exponential reward scales.** DQN's frozen target network [4] prevents moving-target in-

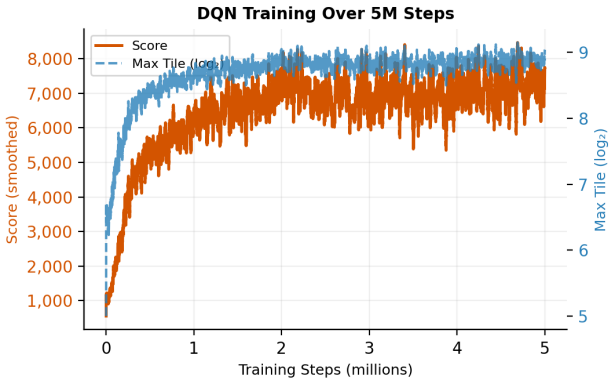


**Figure 3:** DQN training dynamics over 5M steps. The left axis shows smoothed score (orange); the right axis shows the maximum tile achieved per episode in $\log_2$ scale (blue dashes). Both metrics exhibit steady improvement, with the score growth rate increasing after 2M steps.

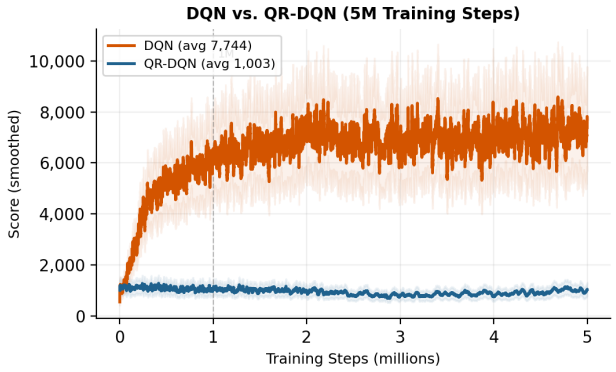


**Figure 4:** Head-to-head comparison of DQN and QR-DQN learning curves over 5M training steps. Despite modeling the full return distribution, QR-DQN plateaus below 1,500 while DQN improves monotonically. Shaded bands indicate $\pm 25\%$ of the smoothed score.

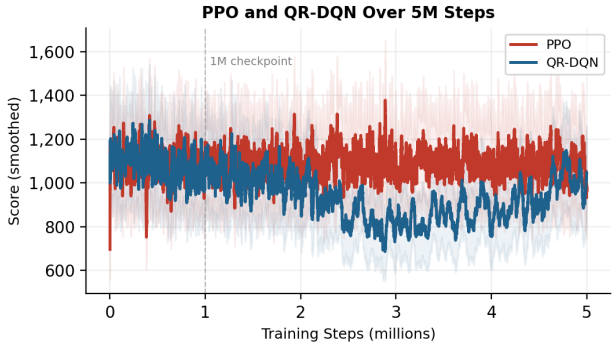


**Figure 5:** Training curves for PPO and QR-DQN over 5M steps. PPO peaks near 1M steps and subsequently degrades ($-8\%$). QR-DQN shows modest improvement (+28%) but remains substantially below DQN.

stability. On 2048's exponentially growing tile values, successive Q-value estimates can differ by orders of magnitude as the policy improves. A synchronized (non-frozen) target would induce oscillation precisely at the high-value transitions most critical for learning.

**Entropy regularization in sparse-reward settings.** SAC's automatic temperature tuning [8] targets entropy $0.4 \cdot \log|\mathscr{A}| \approx 0.55$ nats. In 2048, the majority of board states yield low reward regardless of action. Maintaining high entropy promotes uniform action selection over these low-value states, diverting update capacity away from the greedy exploitation of the few high-value trajectories that drive score improvement.

**Clipping constraints under heavy-tailed rewards.** PPO's $\varepsilon$=0.2 clip [6] bounds the magnitude of policy updates per iteration. In 2048, a single merge of two 1024-tiles yields 1024 reward units while hundreds of preceding moves yield negligible reward. The same clipping that prevents destructive updates under smooth reward surfaces also limits the gradient magnitude needed to propagate value from these rare, high-reward merges back through the trajectory.

### 5.5 DQN Training Dynamics

Figure 3 shows the joint evolution of DQN's average score and maximum tile achieved (in $\log_2$ scale) over 5M training steps. Both metrics improve monotonically, with score growth accelerating after approximately 2M steps as the agent learns to chain high-value merges. The $\log_2$ tile metric indicates that DQN consistently reaches tile 512 ($2^9$) after 1M steps and tile 1024 ($2^{10}$) after 3M steps, with sporadic 2048 ($2^{11}$) episodes appearing in the final million steps.

### 5.6 DQN vs. QR-DQN: Distributional RL Comparison

QR-DQN models the full return distribution via quantile regression [5], providing a richer learning signal than DQN's scalar Q-estimates. Despite this theoretical advantage, QR-DQN underperforms DQN on 2048, as shown in Figure 4.

DQN's score increases monotonically to an average of 7,744 at 5M steps, while QR-DQN plateaus near 1,000. Two factors may account for this gap. First, the $4\times50$=200-dimensional output head introduces higher variance in gradient estimates relative to DQN's compact 4-dimensional output. Second, the quantile Huber loss [5] penalizes all quantile crossings equally; however, 2048's return distribution is heavy-tailed, with a small number of episodes producing scores 10–40$\times$ the median. These outlier episodes contain the most informative learning signal, and distributing them across 50 quantiles attenuates their gradient contribution.

### 5.7 On-Policy Degradation

Figure 5 isolates the training curves of PPO and QR-DQN to highlight their contrasting behaviors from the 1M checkpoint onward.

PPO's degradation is attributable to the interaction be-

**Table 3:** Hunt mode results using the 1M checkpoint with $\varepsilon$=0.05 exploration. DQN is the only agent to reach tile 2048; PPO collapses in under 200 attempts.

| Agent | Attempts | Best Tile | Best Score | Verdict |
|---|---|---|---|---|
| DQN | 3,902 | **2048** | **21,572** | Only success |
| SARSA | 3,000 | 1024 | 12,300 | 120-param ceiling |
| A2C | 3,000 | 512 | 7,844 | Stalls post-512 |
| SAC | 3,000 | 256 | 3,648 | Entropy hurts late |
| PPO | 193 | 64 | 520 | Collapses early |

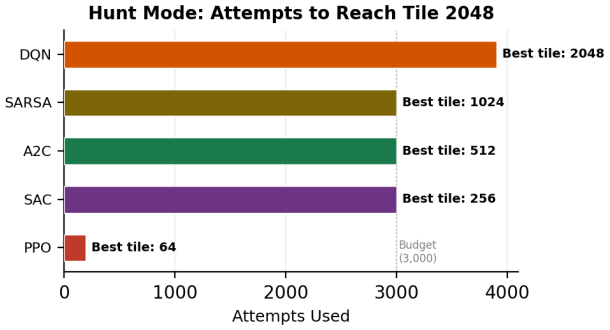


**Figure 6:** Hunt mode: number of attempts used (horizontal axis) and best tile achieved (labels) per agent. The dotted line marks the 3,000-attempt budget. DQN exceeded the budget but reached tile 2048; PPO collapsed in 193 attempts. QR-DQN is excluded as hunt mode was not run for this agent.

tween its clipped objective and 2048's non-stationary reward landscape. Early in training, the policy visits diverse board states; the clipping mechanism preserves this diversity. As training continues, the policy converges to a narrow trajectory distribution, and the clipped updates reinforce these trajectories rather than exploring alternatives. Without a replay buffer to maintain trajectory diversity, the policy collapses to degenerate looping behavior.

### 5.8 Hunt Mode (1M Checkpoint, Unlimited Attempts)

Each agent plays with stochastic action selection ($\varepsilon$=0.05) using its 1M-step checkpoint until reaching tile 2048 or exhausting 3,000 attempts. Table 3 and Figure 6 present the results.

DQN is the only agent to reach tile 2048, requiring 3,902 attempts (0.026% success rate). Hunt mode reveals that deterministic greedy policies can enter infinite loops at board states where Q-values are poorly estimated; the 5% exploration probability breaks these cycles by occasionally selecting alternative actions. PPO reaches only tile 64 within 193 attempts, a result below uniform random play, indicating that its on-policy training produced a policy narrowly specialized to the trajectory distribution seen during training and unable to generalize under stochastic action selection.

**Table 4:** GRPO-trained LLM performance after 500 training steps over 2,000 prompts. The 3× increase in model parameters yields a 3.3× score improvement.

| Model | Avg Score | Max Tile | Params | ms/move |
|---|---|---|---|---|
| Qwen2.5-1.5B | **2,348** | **256** | 1.8B | 1,500 |
| Qwen2.5-0.5B | 704 | 64 | 494M | 1,500 |

### 5.9 RLVR Results: GRPO-Trained LLMs

#### *5.9.1 What GRPO Is and How It Was Applied.*

Group Relative Policy Optimization [9] is a critic-free policy gradient method originally developed for mathematical reasoning. Rather than maintaining a separate value network to estimate baselines, GRPO generates a *group* of $G$ completions for each prompt and normalizes advantages relative to that group's reward distribution (Eq. 1). Eliminating the critic reduces peak memory consumption compared to standard actor-critic methods, enabling fine-tuning on consumer-grade GPUs with limited VRAM.

In this work, GRPO is applied to a fundamentally different domain: spatial strategy in 2048. Each training "prompt" consists of an ASCII-rendered board state, and the model must output a structured XML response containing a chain-of-thought reasoning trace followed by a directional action. The verifiable reward comes directly from the game environment, avoiding any reliance on a reward model or human feedback.

We fine-tune two Qwen2.5-Instruct models: a 0.5B and a 1.5B variant. Both are loaded in 4-bit NF4 quantization via Unsloth [17] with LoRA adapters ($r$=16, $\alpha$=16), keeping memory usage below 6 GB VRAM. Training runs for 500 GRPO optimizer steps over a dataset of 2,000 board-state prompts for 3 epochs, generating $G$=4 completions per prompt. Total training time is approximately 41 minutes (0.5B) and 76 minutes (1.5B) on an RTX 4060 Laptop GPU.

#### *5.9.2 Reward Schedule and Cold-Start Mitigation.*

A naïve game-reward-only objective causes cold-start collapse: the model fails to produce parseable output and receives all-zero rewards, causing the policy to diverge immediately. We use a three-stage curriculum. Stage 1 (steps 0–100) rewards only format compliance (XML tag structure, weight 0.5). Stage 2 (steps 100–200) adds a direction validity reward (weight 0.5), penalizing moves outside {UP, DOWN, LEFT, RIGHT}. Stage 3 (steps 200–300) introduces the game reward ($\Delta$score / max_tile, weight 2.0). Length and quality bonus terms were removed after the model learned to reward-hack them by generating filler tokens between the thinking and answer tags.

Table 4 compares the two GRPO-trained LLM agents.

The 3× parameter increase (0.5B → 1.5B) yields a 3.3× score improvement (704 → 2,348), as shown in Figure 7. While only two data points prevent strong claims about the functional form of this relationship, the magnitude of the gain

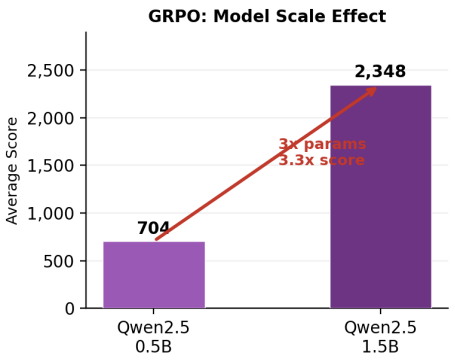


**Figure 7:** GRPO scaling: 3× more parameters (0.5B → 1.5B) yield a 3.3× improvement in average score. Only two data points are available; the functional form of this relationship remains to be established with additional model sizes.

suggests that model capacity is a primary lever for RLVR on spatial tasks.

#### *5.9.3 Training Dynamics.*

Figure 8 decomposes the GRPO training process into three diagnostic panels. Panel (a) shows the total reward trajectory for both models over the logged 500 optimizer steps. The 1.5B model achieves consistently higher cumulative reward and substantially lower variance across the group of $G$=4 completions. High within-group reward variance produces noisy advantage estimates (Eq. 1), degrading gradient quality, so this difference is mechanistically important.

Panel (b) separates the format and direction reward components. Both models learn format compliance (generating syntactically valid XML tags) within approximately 50 training steps, confirming that instruction-tuned base models carry strong structural priors. However, the 0.5B model takes nearly three times as many steps to achieve consistent direction validity, indicating that its capacity is barely sufficient to simultaneously maintain format structure and reason about spatial moves.

Panel (c) shows the KL divergence from the reference policy on a logarithmic scale. The 0.5B model exhibits order-of-magnitude larger KL spikes in the first 100 steps. Large KL spikes indicate that GRPO is forcing aggressive policy updates because the base model is poorly initialized for this task at this scale. The KL penalty ($\beta$=0.04 in Eq. 1) prevents divergence but constrains policy shift per step, imposing a capacity tax on the smaller model.

#### *5.9.4 Critical Analysis of GRPO on Spatial Tasks.*

Several structural limitations of GRPO become apparent when applied outside its original mathematical reasoning domain.

**Token-space vs. state-space reasoning.** GRPO optimizes token-level log-probabilities, but spatial reasoning requires consistent modeling of a 4×4 grid where position relationships are not preserved in token order. The ASCII encoding forces the model to learn board geometry implicitly from positional labels ([row][col]), a fundamentally different inductive bias than the convolutional structure exploited by classical agents.

**Data efficiency.** Five hundred training steps with $G$=4 completions each expose the model to approximately 2,000 board states. In contrast, DQN processes 5M transitions with a 100K-transition replay buffer, revisiting each state many times. This 2,500× sample ratio is the primary quantitative explanation for the 3.3× score gap between GRPO 1.5B (2,348) and DQN (7,744).

**Reward hacking.** The length and quality bonus terms were exploited by the 0.5B model, which generated verbose filler text between the `<think>` and `<answer>` tags to maximize sequence-length rewards without improving game performance. This is a canonical RLVR failure mode.

**Scalability outlook.** The 3.3× score gain from a 3× parameter increase is encouraging, but both models are limited to tile 256. Reaching tile 2048 via GRPO would likely require models in the 7B–13B range with longer training horizons, hardware unavailable in this study.

#### *5.9.5 Chain-of-Thought Reasoning.*

The 1.5B model produces interpretable move explanations during interactive evaluation. For example, given a board with 256 anchored in the bottom-right corner, we observed the following output:

```
<think> 256 anchored bottom-right -
good.
Gradient:  128, 64, 32, 16 intact
along right/bottom edges.
RIGHT merges 4+4 in row 1 (+8).
Avoids scattering corner.
</think> <answer>RIGHT</answer>
```

This reasoning trace mirrors the snake-pattern heuristic used in classical agents: anchor the highest tile in a corner and build a monotonic gradient outward. That the model generates this strategy from 500 training steps on 2,000 prompts is notable.

#### *5.9.6 Spatial Reasoning Failure.*

The 0.5B model reveals a fundamental limitation observed during interactive evaluation: it pattern-matches on token proximity rather than grid geometry. When tiles 4 appear at positions [0][1] and [1][0], the model incorrectly claims "4 at [0][1] and [1][0] are adjacent; LEFT merges them into 8." In reality, they are in different rows; LEFT moves each to column 0 independently with no merge. This type of spatial reasoning failure, where the model confuses token adjacency in the serialized representation with spatial adjacency on the board, is a qualitative observation from interactive testing. A systematic categorization of failure modes across a larger sample would strengthen this finding and is left for future work.

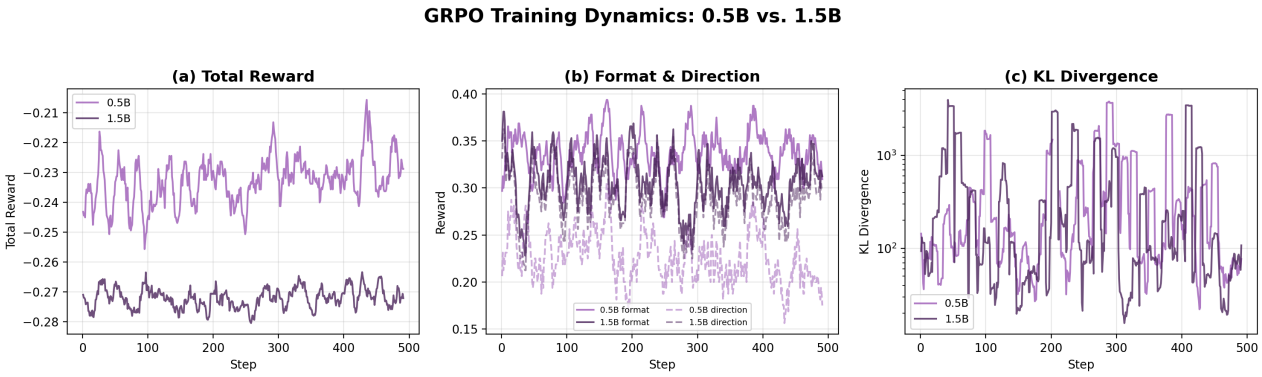


**Figure 8:** GRPO training dynamics for the 0.5B and 1.5B models across 500 optimizer steps. (a) Total reward: 1.5B converges higher with lower within-group variance. (b) Format and direction reward: format compliance is learned quickly by both models, but valid direction prediction is significantly slower for 0.5B. (c) KL divergence (log scale): 0.5B exhibits order-of-magnitude larger policy deviations early in training, reflecting capacity-constrained optimization.

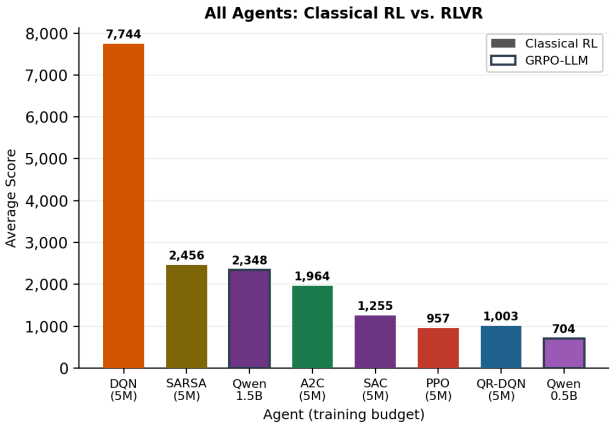


**Figure 9:** Average score comparison across all eight agents. Classical agents are shown with solid fills; GRPO-LLM agents are outlined. DQN at 5M steps achieves the highest score (7,744), while GRPO 1.5B (2,348) outperforms three of the five deep classical agents.

### 5.10 Classical RL vs. RLVR Comparison

Figure 9 directly compares all eight agents. Table 5 provides a dimension-by-dimension analysis.

The 3.3× score gap between DQN (7,744) and GRPO 1.5B (2,348) reflects fundamental differences in data efficiency and representation. DQN processes 5M transitions with replay, revisiting each state thousands of times; GRPO sees approximately 2,000 board states. DQN's 16-channel tensor encoding is information-dense, preserving the spatial structure of the board in a form amenable to convolutional processing; the LLM must tokenize and parse ASCII text, introducing representational overhead that scales with sequence length.

However, RLVR provides a unique advantage: *interpretability*. Every GRPO move includes a chain-of-thought explanation that can be inspected, debugged, and corrected. Classical agents are complete black boxes. In applications where explainability is valued (educational tools, human-AI collaboration, safety-critical systems), the 400× latency penalty may be acceptable.

**Table 5:** Head-to-head comparison across seven dimensions. Classical RL dominates on performance and efficiency; RLVR provides unique interpretability through chain-of-thought reasoning.

| Dimension | Classical RL (best) | LLM-RLVR (GRPO) |
|---|---|---|
| Best Avg Score | **7,744** (DQN, 5M) | 2,348 (1.5B) |
| Best Max Tile | **2048** (DQN, hunt) | 256 (1.5B) |
| Parameters | 120–430K | 494M–1.8B |
| Inference | **<4 ms/move** | 1,500 ms/move |
| Training data | 5M env steps (∼2 hrs) | 2,000 prompts (∼76 min) |
| Training compute | ∼2 hrs (RTX 4060) | ∼76 min (same GPU) |
| Interpretable | No (black box) | **Yes (chain-of-thought)** |
| Observation | **Tensor** (16ch) | ASCII text |
| Action space | Integer $\{0-3\}$ | Natural language token |

### 5.11 Parameter Efficiency

Figure 10 plots average score against parameter count on a logarithmic scale, revealing the non-monotonic relationship between model size and performance.

Semi-gradient SARSA's position (120 parameters, score 2,456) represents the most parameter-efficient configuration in our study. It outperforms four deep agents with 2,750× more parameters and both GRPO-LLM agents with approximately 4,000,000× more parameters. This demonstrates that domain knowledge encoded in hand-crafted features can outperform end-to-end learning from raw observations when the feature space is well-designed for the task (S&B §9.5). However, this advantage depends on expert knowledge of the domain, and SARSA's representational ceiling (the bounded approximation error of linear FA described in §9.4) prevents

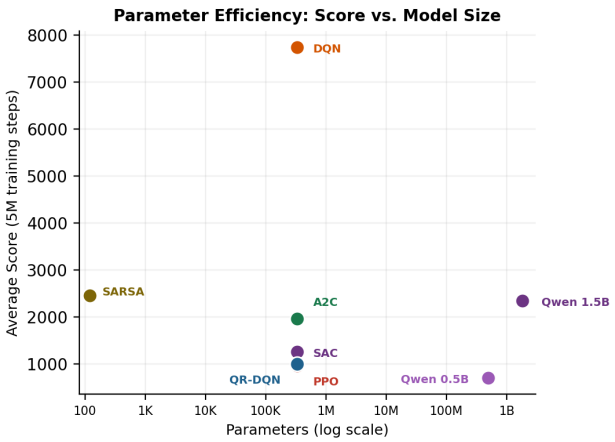


**Figure 10:** Parameter efficiency: average score vs. model size (log scale). Semi-gradient SARSA (120 parameters) outperforms four agents with 2,750× more parameters, and both GRPO-LLM agents with 4,000,000× more parameters. DQN's 330K parameters achieve the best absolute performance. The LLM agents occupy the upper-right quadrant, trading parameter efficiency for interpretability.

it from reaching DQN's performance, which requires the nonlinear function approximation of neural networks.

## 6. Conclusion

We presented a systematic comparison of six classical RL agents and two GRPO-trained LLMs on the 2048 puzzle. Our controlled setup (shared CNN backbone, identical environment, three evaluation tiers: 1M, 5M, hunt mode) isolates algorithmic effects and yields four key findings:

1. **Off-policy replay is decisive.** DQN's 100K replay buffer (S&B §8.2) produces an 8× advantage over on-policy PPO on identical architectures (7,744 vs. 957). Experience replay amplifies sparse rewards that on-policy methods see exactly once and discard.
2. **Features beat deep networks at low data.** Semi-gradient SARSA with 120 hand-crafted parameters outperforms all deep on-policy agents (330K+ parameters), validating Sutton & Barto's guidance (§9.5) that domain-specific features can provide stronger inductive bias than CNNs learning from scratch under sparse-reward conditions; however, this advantage depends on expert knowledge of the domain, which is not freely available in novel environments.
3. **Extended training can degrade on-policy agents.** PPO loses 8% score from 1M→5M steps, developing degenerate move loops. This result indicates that longer training does not universally improve performance on sparse-reward tasks.
4. **LLM scale drives RLVR performance.** preliminary evidence from two model scales (3× parameters → 3.3× higher score) suggests spatial reasoning benefits from model capacity; GRPO uniquely produces interpretable per-move rationale, but spatial reasoning remains a bottleneck at small scales.

Future work includes scaling GRPO to Qwen2.5-3B/7B on an A100 GPU. Because our primary focus was algorithm comparison on identical architectures rather than hyperparameter sweeps, we did not perform a strict ablation study on buffer capacity; therefore, systematic replay buffer ablation (e.g., 10K–500K) to quantify the exact contribution of replay to DQN's dominance remains a logical next step. Additional work includes multi-seed evaluation for statistical significance, and hybrid LLM-MCTS approaches combining interpretability with search efficiency.

## Acknowledgment

This project was completed as part of CS 5180 (Reinforcement Learning) taught by Professor Robert Platt at Northeastern University. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8 GB VRAM). Additionally, AI tools were used to assist in the codebase development, as well as the generic formatting, structuring, and drafting of this manuscript.

## References

[1] Daya Guo et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *Nature*, 645:633–638, 2025. arXiv:2501.12948.

[2] Yuqing Wen et al. Rlvr implicitly incentivizes correct reasoning. *arXiv preprint arXiv:2506.14245*, 2025.

[3] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[5] Will Dabney, Mark Rowland, Marc Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *AAAI Conference on Artificial Intelligence*, 2018.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous

methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937, 2016.

[8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.

[9] Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[10] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of n-tuple networks for the game 2048. In *2014 IEEE Conference on Computational Intelligence and Games*, 2014.

[11] Prady Saligram, Tanvir Bhathal, and Robby Manihani. 2048: Reinforcement learning in a delayed reward environment. *arXiv preprint arXiv:2507.05465*, 2025. Withdrawn for result verification.

[12] Lanxiang Hu et al. lmgame-bench: How good are llms at playing games? *arXiv preprint arXiv:2505.15146*, 2025.

[13] Mark Towers et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[14] Petros Christodoulou. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*, 2019.

[15] Antonin Raffin et al. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[16] Qwen Team. Qwen2.5: A party of foundation models. *arXiv preprint arXiv:2412.15115*, 2024.

[17] Daniel Han and Michael Han. Unsloth. `https://github.com/unslothai/unsloth`, 2024.